Grounded Sequential Decision Processes with Decoupled Process Verification and Adaptive Test-Time Compute for Visual Document Reasoning

First Author[a,b] (first.author@mail.com), Second Author[a]
(second.author@mail.com), Last Author[c] (last.author@mail.com)

Abstract

Multimodal Vision-Language Models (VLMs) have demonstrated strong capabilities in Visual Document Understanding (VDU), yet single-pass generation entangles visual perception, procedural planning, tool execution, and answer synthesis into a single unverified decoding path. This entanglement leaves intermediate errors undetected, leading to ungrounded reasoning and hallucinated responses in visually dense, tabular, and multi-page document QA tasks. This study proposes a grounded sequential decision process framework with decoupled process verification and adaptive test-time computation for visual document reasoning. The framework decomposes complex document QA into five functional agent stages: a Context Agent for spatial encoding and safety filtering, a Planning Agent with explicit plan feasibility gating, an Execution Agent with tool-assisted semantic reasoning, a decoupled Verification Agent evaluating spatial layout loss (D_{layout}) and intermediate trajectory consistency, and a Decision Agent enforcing threshold-based selective abstention ($\perp$).

The framework was evaluated across 24 model-dataset configurations comprising six open-weight VLMs (Qwen2-VL-2B/7B, InternVL2-2B/8B, Phi-3-Vision-128k, PaliGemma-3B) and four benchmark datasets (DocVQA, ChartQA, TableBench, MMLongBench-Doc).

The proposed framework achieves positive accuracy gains in 22 of 24 settings (91.7%), increasing overall mean accuracy from 71.0% (Single-Pass) and 75.3% (Self-Consistency N = 3) to 81.0% (+10.0 percentage point gain over single-pass, $p < 0.001$). On Qwen2-VL-7B, our method reaches 89.8% mean accuracy compared to 83.0% for Standard MACT at slightly lower measured computation and latency in the evaluated setting (2.40 s latency, 7.1 x 10^12 FLOPs). Systematic ablations indicate that removing decoupled verification lowers ordinary accuracy to 70.3% and selective accuracy to 77.6%, while closed-loop feedback yields an 81.34% Correction Success Rate (CSR) over detected defects. Temperature scaling reduces Expected Calibration Error (ECE) to 3.4% (Brier score 0.042), supporting calibrated selective prediction.

Keywords: Visual Document Reasoning, Vision-Language Models, Multi-Agent Collaboration, Decoupled Process Verification, Adaptive Test-Time Compute, Selective Prediction.

1. Introduction

Visual Document Understanding (VDU) requires interpreting information embedded across complex visual, textual, tabular, and graphical layouts (Mathew et al., 2021; Masry et al., 2022; Wu et al., 2025; Ma et al., 2024). Modern vision-language models (VLMs), such as Qwen2-VL (Wang et al., 2025) and InternVL2 (Chen et al., 2025), have advanced multimodal understanding by directly processing high-resolution document images. However, when deployed in safety-critical and decision-support applications—such as financial report analysis, medical record auditing, and legal document processing—standard single-pass VLM inference exhibits notable vulnerabilities. Monolithic generation entangles document perception, sub-task planning, evidence extraction, numerical calculation, and response generation within a single autoregressive decoding trajectory. Consequently, an undetected error at an early stage (e.g., incorrect table coordinate extraction or misread chart axis) propagates through the decoding sequence, yielding plausible yet factually incorrect answers (Jia et al., 2025).

Retrieval-Augmented Generation (RAG) and tool-augmented agentic workflows (e.g., ReAct) have been introduced to ground language models in external evidence and computational tools (Yao et al., 2023b; Jiang et al., 2023). However, their effectiveness in visual document domains is limited by the quality of retrieved visual patches and the lack of explicit process-level verification. Furthermore, test-time scaling techniques such as Self-Consistency (Wang et al., 2023) allocate uniform sampling budgets across all instances without verifying whether intermediate reasoning paths are grounded in document spatial layout. Standard multi-agent collaboration frameworks, such as MACT (Yu

et al., 2026), decompose document QA into planning, execution, and judgment roles, but rely primarily on internal model self-critique without external spatial grounding metrics, gated feasibility checks, or calibrated abstention mechanisms.

To address these challenges, this paper presents a grounded sequential decision framework that integrates role-specialized multi-agent collaboration with decoupled process verification, closed-loop feedback, and adaptive test-time computation. The framework formalizes visual document reasoning as a finite-horizon sequential decision process under partial observability. The architecture consists of five specialized agents: (1) a **Context Agent** (A_{ctx}) that screens input queries for safety and encodes spatial context embeddings; (2) a **Planning Agent** (A_{plan}) that generates structured execution plans subject to pre-execution feasibility gating (G_{plan}); (3) an **Execution Agent** (A_{exec}) that performs tool-assisted semantic reasoning (OCR, table parsing, arithmetic calculation) verified by an execution integrity gate (G_{exec}); (4) a decoupled **Verification Agent** (A_{ver}) that independently audits intermediate trajectories against document spatial layout loss (D_{layout}) and triggers closed-loop feedback re-execution; and (5) a **Decision Agent** (A_{dec}) that fuses confidence signals (C_{fused}) and enforces threshold-based selective abstention ($\perp$).

We evaluate the framework across 24 model-dataset configurations using six open-weight VLMs and four benchmark datasets. The primary contributions of this work are summarized as follows:

- We introduce the **Verification-Driven Error-Conditioned Compute Allocation Policy (VEC-CAP)**, a test-time inference control mechanism that conditions specialized agentic recovery actions ($a_{\text{re-ground}}, a_{\text{re-plan}}, a_{\text{re-exec}}, a_{\text{secondary_verify}}, a_{\text{abs}}$ on a 5-element diagnosed multimodal error state vector $e_t = (e_{\text{layout}}, e_{\text{plan}}, e_{\text{exec}}, e_{\text{judg}}, e_{\text{ans}})$.
- We implement a five-agent grounded decision pipeline ($A_{\text{ctx}}, A_{\text{plan}}, A_{\text{exec}}, A_{\text{ver}}, A_{\text{dec}}$) incorporating explicit pre-execution feasibility gates (G_{plan}), execution integrity checks (G_{exec}), and spatial layout loss calculation (D_{layout}) to audit evidence groundedness independently of candidate answer generation.

- We demonstrate that error-conditioned compute allocation achieves higher reasoning accuracy (+5.7 percentage points over Self-Consistency $N = 3$) while expending 5.3% fewer FLOPs (7.1×10^{12} FLOPs) compared to uniform path sampling.
- We evaluate the framework across 24 model-dataset configurations, providing gate ablation studies, validation-based temperature calibration (ECE 3.4%, $T = 1.42$), selective risk profiles, manual diagnostic audit failure analysis ($N = 340$, Cohen's $\kappa = 0.86$), and Holm-Bonferroni adjusted statistical significance tests ($p_{\text{adj}} < 0.01$).

2. Related Work

This section reviews the foundational research fields upon which the proposed framework is established: Visual Document Understanding, vision-language architectures, multi-agent collaboration, test-time computation scaling, process verification, confidence calibration, selective prediction, and multi-agent document reasoning. It concludes with a synthesis of the research gap and spatial positioning relative to MACT.

2.1. Visual Document Understanding

Visual Document Understanding (VDU) addresses the automated interpretation of information expressed through the joint textual, visual, structural, and spatial modalities of documents (Mathew et al., 2021). Early VDU architectures relied heavily on sequential OCR and layout analysis pipelines, in which optical character recognition engines extracted text tokens and separate structural classifiers inferred bounding boxes (Huang et al., 2022). While such pipelines provided explicit textual and spatial representations, errors committed during initial character extraction or layout segmentation propagated irreversibly to downstream language understanding modules.

To mitigate pipeline error propagation, end-to-end multimodal document models were introduced. Architectures such as LayoutLMv2 (Xu et al., 2021),

UDoc (Gu et al., 2022), StructText (Li et al., 2021), ERNIE-Layout (Peng et al., 2022), Pix2Struct (Lee et al., 2023), and Donut (Kim et al., 2022) mapped raw document visual features and pixels directly to target sequences. Specialized visual reasoning architectures were further extended to tabular and graphical structures, including Table Transformer (Smock et al., 2021), DePlot (Liu et al., 2023a), MatCha (Methani et al., 2023), ChartOCR (Zhou et al., 2021), PlotQA (Pasupat & Liang, 2020), and TableLlama (Zhang et al., 2024), alongside multimodal document retrieval mechanisms (F'evry et al., 2024; Zhao et al., 2024; Chen et al., 2024). Subsequently, specialized VDU benchmarks were established to evaluate distinct facets of document reasoning:

- **DocVQA** (Mathew et al., 2021): Evaluates visual question answering over single-page and multi-page document images, requiring precise spatial localization and textual extraction measured via Average Normalized Levenshtein Similarity (ANLS).
- **ChartQA** (Masry et al., 2022): Tests visual reasoning over graphical plots, requiring models to extract data series from visual elements and perform logical synthesis.
- **TableBench** (Wu et al., 2025): Assesses multi-step tabular reasoning over complex structured tables, evaluating arithmetic cell operations and structural relationships.
- **MMLongBench-Doc** (Ma et al., 2024): Evaluates long-context multimodal reasoning across multi-page documents where evidence is distributed across heterogeneous pages and non-contiguous visual regions.

Despite significant progress across these benchmarks, monolithic VDU systems process visual perception, structural interpretation, and textual reasoning within a single decoding path, leaving intermediate extraction errors undetected.

2.2. Vision-Language Models for Document Reasoning

The advent of general-purpose Vision-Language Models (VLMs) has transformed visual document processing. Modern open-weight architectures, such

as Qwen2-VL (Wang et al., 2025), InternVL2 (Chen et al., 2025), LLaVA (Liu et al., 2023b), InstructBLIP (Dai et al., 2023), MiniGPT-4 (Zhu et al., 2023), CogVLM (Wang et al., 2024b), PaliGemma (Beyer et al., 2024), and Phi-3 Vision (Abdin et al., 2025), incorporate dynamic-resolution visual encoders and multimodal projection layers that map high-resolution document images directly into language model embedding spaces.

Formally, given a document page image $I \in \mathbb{R}^{H \times W \times 3}$ and a natural-language query q, a monolithic VLM computes the conditional probability of generating answer sequence $y = (y_1, y_2, \ldots, y_T)$ via autoregressive factorization:

$$P_\theta(y \mid I, q) = \prod_{t=1}^{T} P_\theta(y_t \mid y_{<t}, E_{\text{visual}}(I), E_{\text{text}}(q)), \tag{1}$$

where $E_{\text{visual}}(\cdot)$ represents the visual feature extraction network, $E_{\text{text}}(\cdot)$ denotes the text query tokenizer, and θ represents the parameters of the multimodal model backbone.

Although Equation (1) enables impressive zero-shot performance across standard visual QA tasks, monolithic autoregressive decoding treats perception, plan formation, evidence extraction, and numerical execution as an unverified single-pass generation. When processing visually dense or multi-page documents, subtle errors in early token generation (e.g., misreading a table header coordinate) distort subsequent conditional probabilities $P_\theta(y_t \mid y_{<t}, \cdot)$, yielding hallucinated or factually inconsistent answers (Jia et al., 2025). This vulnerability highlights the necessity of decomposing monolithic VLM generation into structured, verifiable reasoning stages.

2.3. Multi-Agent Reasoning

Multi-agent reasoning has emerged as an effective paradigm for solving complex tasks by distributing problem-solving responsibilities across specialized language-model agents (Li et al., 2023; Hong et al., 2024; Wu et al., 2024; Liu et al., 2024). In tool-augmented agentic frameworks such as ReAct (Yao et al., 2023b), Toolformer (Schick et al., 2023), Gorilla (Patil et al., 2023), Reflexion (Shinn et al., 2023), Self-Refine (Madaan et al., 2023), and StarCoder (Li et al.,

2024), interleaving reasoning traces with external tool executions enables models to retrieve evidence, query external databases, and perform deterministic arithmetic operations.

Multi-agent architectures can be broadly categorized into three structural paradigms:

1. *Sequential Pipelines:* Pass intermediate textual outputs from one specialized agent to the next, decomposing complex tasks into ordered subproblems (Hong et al., 2024).
2. *Parallel & Debate Architectures:* Generate multiple candidate solutions independently and aggregate their outputs via majority voting or inter-agent debate (Du et al., 2024).
3. *Hierarchical Orchestration:* Employ a manager agent to dynamically delegate sub-tasks to specialized worker agents and synthesize final outputs (Wu et al., 2024).

Despite their operational advantages, standard multi-agent collaboration frameworks exhibit structural failure modes. In sequential pipelines, unverified intermediate errors committed by upstream agents propagate downstream, causing compounding execution failures. Parallel debate systems increase computational cost linearly and suffer from correlated errors when agent instances share the same underlying VLM backbone (Du et al., 2024). Effective multi-agent visual document reasoning therefore requires component-level process verification and strict pre-execution gating mechanisms.

2.4. Test-Time Scaling and Adaptive Inference

Test-time scaling improves reasoning accuracy by allocating additional computational budget during inference without retraining model weights (Snell et al., 2024; Brown et al., 2024; Zelikman et al., 2024). Building upon Chain-of-Thought prompting (Wei et al., 2022), tree-search paradigms such as Tree of Thoughts (Yao et al., 2023a) and Graph of Thoughts (Besta et al., 2024) explore structured search spaces during generation. A foundational approach is

Self-Consistency (Wang et al., 2023), which samples N independent chain-of-thought reasoning paths $\tau_1, \tau_2, \ldots, \tau_N \sim P_\theta(\tau \mid I, q)$ and selects the majority answer.

More recent work models test-time computation as an optimal resource allocation problem over candidate reasoning paths. Let $\mathcal{T}$ denote the space of generated reasoning trajectories, and let $V(\tau)$ represent a process verification signal evaluating path quality. The verifier-guided test-time optimization objective is formulated as:

$$\tau^* = \arg\max_{\tau \in \mathcal{T}} V(\tau) \quad \text{subject to} \quad \sum_{t=1}^{|\tau|} c_t(k_t) \leq B_{\text{infer}}, \tag{2}$$

where $c_t(k_t)$ represents the computational cost (FLOPs or token volume) expended at step t, and B_{infer} denotes the overall inference budget constraint.

While fixed-budget sampling methods (e.g., Self-Consistency $N = 3$) expend additional computation uniformly across all instances regardless of query difficulty, adaptive inference dynamically adjusts candidate sampling, tool invocations, and verification retries based on intermediate confidence signals (Snell et al., 2024). In visual document reasoning, allocating test-time compute adaptively according to verified layout loss prevents unnecessary computation on simple document queries while dedicating retries to ambiguous tabular or multi-page instances.

2.5. Process Verification, Confidence Calibration, and Selective Prediction

Process verification evaluates intermediate reasoning steps rather than relying solely on final-answer correctness (Lightman et al., 2023; Wang et al., 2024c,a; Uesato et al., 2022). Outcome supervision evaluates final answer accuracy $R(y, y_{\text{ground_truth}})$, whereas process supervision evaluates intermediate state transitions ($s_t \to s_{t+1}$), providing localized error signals to intercept flawed reasoning before final answer synthesis (Jia et al., 2025; Park et al., 2025).

To ensure that verification signals reliably guide execution and response control, confidence estimates must be empirically calibrated (Guo et al., 2017). Let $\hat{P} \in [0, 1]$ denote the predicted confidence score and let $Y \in \{0, 1\}$ denote

empirical correctness. Expected Calibration Error (ECE) quantifies calibration quality by partitioning predicted probabilities into M equal-width bins B_m:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|, \tag{3}$$

where N is the total number of evaluation samples, $\mathrm{acc}(B_m)$ represents average empirical accuracy within bin B_m, and $\mathrm{conf}(B_m)$ denotes the average predicted confidence in bin B_m.

Calibrated confidence enables safe selective prediction, allowing models to abstain ($\perp$) when confidence falls below a safety threshold $\theta_{\text{threshold}}$ (Chow, 1970; Geifman & El-Yaniv, 2017; Angelopoulos & Bates, 2023; Tian et al., 2023). Given a target coverage level $\rho \in (0, 1]$, the selective prediction objective minimizes selective risk $\mathrm{Risk}(\rho)$:

$$\mathrm{Risk}(\rho) = \mathbb{E}_{(I,q,y)\sim\mathcal{D}} \left[\ell(\hat{y}, y) \mid g(I, q) = 1\right], \tag{4}$$

where $\ell(\hat{y}, y)$ represents answer loss, and $g(I, q) \in \{0, 1\}$ is a selection function indicating whether the system answers ($g = 1$) or abstains ($g = 0$, returning $\perp$) at coverage ρ. Integrating process verification, calibrated confidence fusion, and selective prediction allows visual document reasoning systems to defer ungrounded queries, suppressing hallucinated responses in safety-critical domains.

2.6. Multi-Agent Visual Document Reasoning: MACT

MACT introduces a multi-agent collaboration framework with agent-wise adaptive test-time scaling specifically designed for visual document understanding (Yu et al., 2026). MACT decomposes visual document QA into four specialized agent roles: Planning, Execution, Judgment, and Answer agents. It applies agent-wise scaling mechanisms—generating multiple candidate plans for the planning agent and adjusting reasoning effort for execution—and incorporates mixed reward modeling to align individual agent objectives with end-to-end answer correctness.

While MACT establishes the utility of multi-agent role specialization and agent-wise scaling in VDU, several operational limitations remain:

Table 1: Comparison of representative approaches across key capabilities relevant to visual document reasoning.

Method / Framework	VDU Domain	Feasibility Control	Process Verification
Monolithic VLMs (Wang et al., 2025; Chen et al., 2025)	✓	×	×
ReAct-Doc (Yao et al., 2023b)	✓	×	×
Self-Consistency (Wang et al., 2023)	✓	×	×
MentalLLM-Style (Mukta et al., 2025)	✓	×	×
Standard MACT (Yu et al., 2026)	✓	×	×
Proposed Framework	✓	✓	✓

1. *Internal Judgment Dependency:* MACT's judgment agent relies primarily on internal language-model self-critique without external spatial layout loss modeling (D_{layout}) or spatial coordinate verification against document bounding boxes.
2. *Absence of Pre-Execution Feasibility Control:* MACT executes generated plans directly, evaluating errors post-execution rather than enforcing pre-execution feasibility gates ($G_{\text{plan}}, G_{\text{exec}}$) to reject flawed plans before tool invocation.
3. *Unconditional Answer Generation:* MACT forces final answer generation across all queries regardless of uncertainty, lacking an explicit confidence-aware selective prediction mechanism to return selective abstention ($\perp$) on ambiguous queries.

Table 1 summarizes representative visual document reasoning approaches across eight core capabilities. Existing paradigms address visual perception, tool interaction, or sampling-based test-time scaling as isolated objectives. Monolithic VLMs and ReAct-Doc lack process verification and calibration. Self-Consistency scales compute uniformly without verifying layout grounding. Standard MACT introduces role specialization and adaptive scaling but lacks decoupled spatial layout verification (D_{layout}), pre-execution feasibility gating ($G_{\text{plan}}, G_{\text{exec}}$), and calibrated threshold abstention ($\perp$).

The primary research gap addressed in this work is the absence of a unified

sequential decision framework that integrates:

- pre-execution feasibility and integrity gating to prevent compounding error propagation;
- decoupled external process verification using spatial layout loss (D_{layout}) to audit evidence groundedness independently of answer generation;
- adaptive verification-driven compute allocation that dynamically manages retries and tool calls; and
- calibrated fused-confidence thresholding that enforces selective abstention ($\perp$) under high uncertainty.

This gap motivates the five-agent closed-loop methodology detailed in Section 3.

3. Proposed Methodology

This section presents the proposed framework for grounded visual document reasoning. Building on the research gap identified in Section 2.6, the framework integrates role-specialized multi-agent reasoning with explicit process verification, corrective feedback, confidence-aware decision making, and adaptive test-time computation. The central motivation is to avoid treating visual document question answering as a single-pass generation problem in which errors introduced during early reasoning stages are propagated directly to the final answer. Instead, the proposed framework decomposes the reasoning process into distinct functional stages and enables intermediate results to be evaluated before they are used for subsequent decisions.

The proposed architecture consists of five specialized agents: a Context Agent, a Planning Agent, an Execution Agent, a Verification Agent, and a Decision Agent. The agents collaboratively transform the input document and query into a grounded final decision. In addition, feasibility and integrity checks are introduced between critical reasoning stages so that infeasible plans and execution failures can be detected before they propagate through the reasoning

trajectory. The Verification Agent provides process-level feedback, allowing the framework to revise earlier reasoning stages when necessary. Finally, an adaptive computation mechanism determines whether additional reasoning should be performed based on the current state of the reasoning process and its associated confidence.

3.1. Problem Formulation

We formulate visual document reasoning as a grounded sequential decision problem in which a system must produce a reliable answer to a natural-language query while reasoning over heterogeneous visual and textual evidence. Let a visual document be represented as an ordered sequence of N page images,

$$D = \{I_1, I_2, \ldots, I_N\}, \qquad I_k \in \mathbb{R}^{H \times W \times 3}, \tag{5}$$

where each page I_k may contain textual content, tables, charts, figures, mathematical expressions, and spatial layout structures. Let $q \in \mathcal{Q}$ denote a natural-language query associated with the document. The query may require factual extraction, visual interpretation, numerical calculation, or evidence synthesis across multiple document regions or pages.

Let $\mathcal{A}$ denote the answer space. In addition to conventional textual or numerical answers, the framework allows a selective abstention decision $\perp$ when the available evidence is insufficient, inconsistent, or associated with low confidence. The desired output is therefore

$$\hat{a} \in \mathcal{A} \cup \{\perp\}. \tag{6}$$

A conventional monolithic vision-language model formulates the task as direct conditional generation:

$$\hat{a} = \arg\max_{a \in \mathcal{A}} P_\theta(a \mid D, q), \tag{7}$$

where θ denotes the parameters of the underlying vision-language model. For an autoregressive decoder, this probability can be decomposed as

$$P_\theta(a \mid D, q) = \prod_{t=1}^{T} P_\theta(a_t \mid a_{<t}, D, q), \tag{8}$$

where a_t represents the t-th generated token and $a_{<t}$ denotes the preceding token sequence.

Although this formulation provides a unified mechanism for multimodal reasoning, it does not explicitly represent the intermediate operations required for complex visual document questions. In particular, document reasoning may require identifying relevant regions, decomposing the query into sub-tasks, extracting visual or textual evidence, executing numerical or logical operations, verifying intermediate results, and finally synthesizing the answer. An error introduced during an early reasoning operation can consequently influence subsequent operations through the autoregressive dependency structure.

To explicitly represent this procedural nature, we define a reasoning trajectory as

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_T\}, \tag{9}$$

where s_t denotes the reasoning state at step t and a_t denotes an action performed at that state. An action may correspond to a planning operation, document-region inspection, tool execution, evidence extraction, verification operation, or answer-selection decision. The transition between reasoning states is represented by

$$s_{t+1} \sim P(s_{t+1} \mid s_t, a_t, D, q). \tag{10}$$

The reasoning process is therefore modeled as a finite-horizon sequential decision problem under partial observability. The complete decision process can be expressed as

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle, \tag{11}$$

where $\mathcal{S}$ denotes the latent reasoning-state space, $\mathcal{A}$ represents the available reasoning actions, $\mathcal{P}$ is the state-transition model, $\mathcal{R}$ is the process-reward function, Ω is the observation space, $\mathcal{O}$ is the observation model, and $\gamma \in (0, 1]$ is the discount factor.

Within this formulation, the reasoning system does not treat the final answer as the only quantity of interest. Each intermediate state may be associated with a process-quality signal,

$$r_t = R(s_t, a_t, D, q), \tag{12}$$

which reflects the quality of the corresponding reasoning operation. Such process-level evaluation is particularly important for visual document reasoning because an apparently plausible final answer may originate from an incorrect document region, an erroneous table extraction, an invalid calculation, or an unsupported intermediate inference.

The proposed framework therefore introduces explicit verification of intermediate reasoning states. Let

$$v_t = V(s_t, a_t, D, q) \tag{13}$$

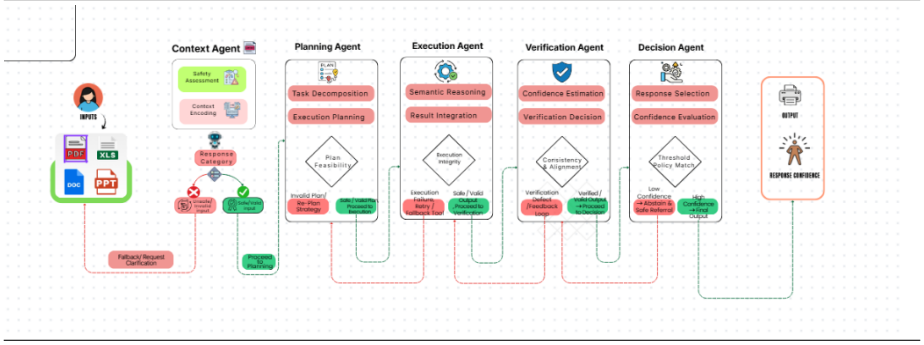
denote the verification signal associated with step t. The verification signal is used to determine whether the current reasoning state is sufficiently supported by the available document evidence or whether the system should revise the corresponding reasoning step.

When the verification signal indicates insufficient reliability, an additional reasoning or correction step may be activated. Let k_t denote the amount of computation allocated at reasoning stage t. The total inference budget can then be expressed as

$$C(\tau) = \sum_{t=1}^{T} c_t(k_t), \tag{14}$$

where $c_t(k_t)$ denotes the computational cost associated with allocating k_t units of inference computation to stage t. Rather than assigning an identical

Figure 1 System architecture of the grounded five-agent framework for visual document reasoning, illustrating the sequential processing and closed-loop feedback pathways across the Context Agent (A_{ctx}), Planning Agent (A_{plan}), Execution Agent (A_{exec}), Verification Agent (A_{ver}), and Decision Agent (A_{dec}).



computation budget to every query, the framework seeks to allocate additional computation when the current reasoning state exhibits uncertainty, inconsistency, or verification failure.

The overall objective is therefore to identify a final decision that jointly considers answer quality, process reliability, and inference cost:

$$\hat{a} = \arg \max_{a \in \mathcal{A} \cup \{\perp\}} \mathbb{E}\left[R_{\text{final}}(a) + \lambda R_{\text{process}}(\tau) - \eta C(\tau) \mid D, q\right], \tag{15}$$

where R_{final} measures the quality of the final decision, R_{process} captures the quality of the intermediate reasoning trajectory, $C(\tau)$ represents the computational expenditure, and $\lambda, \eta \geq 0$ control the relative importance of reasoning reliability and computational efficiency.

This formulation establishes the basis for the proposed framework. The following subsections describe the multi-agent architecture, verification and corrective feedback mechanism, adaptive test-time computation, and confidence-aware final decision process.

3.2. Five-Agent Architecture

The proposed framework employs five specialized agents to decompose the visual document reasoning process into complementary functional stages. The

architecture is designed to separate contextual understanding, reasoning-plan construction, execution, process verification, and final decision making. This separation enables intermediate reasoning states to be inspected and corrected rather than allowing errors to propagate directly from an early stage to the final answer.

Let the set of specialized agents be

$$\mathcal{G} = \{A_{\text{ctx}}, A_{\text{plan}}, A_{\text{exec}}, A_{\text{ver}}, A_{\text{dec}}\}, \tag{16}$$

where A_{ctx}, A_{plan}, A_{exec}, A_{ver}, and A_{dec} denote the Context, Planning, Execution, Verification, and Decision Agents, respectively.

Given a document–query pair (D, q), the Context Agent first constructs a grounded representation of the available document information. The Planning Agent then uses this representation to construct a sequence of operations required to answer the query. The Execution Agent performs the planned operations and produces intermediate evidence and results. These results are subsequently examined by the Verification Agent, which evaluates their consistency and evidential support. Finally, the Decision Agent integrates the verified reasoning state and determines whether to return a final answer or abstain.

The overall forward interaction is represented as

$$(D, q) \to A_{\text{ctx}} \to A_{\text{plan}} \to A_{\text{exec}} \to A_{\text{ver}} \to A_{\text{dec}} \to \hat{a}. \tag{17}$$

The architecture is not restricted to a strictly one-directional pipeline. Verification results can trigger corrective feedback when the current reasoning state is judged unreliable. In this case, the process may return to the Planning or Execution Agent, depending on the type of detected failure. The resulting architecture can therefore be represented as a closed-loop process:

$$A_{\text{plan}} \to A_{\text{exec}} \to A_{\text{ver}} \to \begin{cases} A_{\text{dec}}, & \text{if the reasoning is accepted,} \\ A_{\text{plan}}, & \text{if replanning is required,} \\ A_{\text{exec}}, & \text{if re-execution is required.} \end{cases} \tag{18}$$

3.2.1. Context Agent

The Context Agent, A_{ctx}, serves as the front-end component of the framework. Its primary role is to establish a grounded representation of the document and query before planning and execution take place. Given (D, q), it produces

$$c = A_{\text{ctx}}(D, q). \tag{19}$$

The contextual representation may contain relevant textual evidence, visual regions, document structure, page-level information, and spatial relationships required for downstream reasoning. It can be represented as

$$c = \{E_{\text{text}}, E_{\text{visual}}, E_{\text{struct}}, E_{\text{spatial}}\}. \tag{20}$$

The Context Agent therefore establishes the evidence-conditioned state from which the remaining reasoning process is performed. Importantly, it does not directly determine the final answer. Its role is to provide a grounded evidence context that can subsequently be interpreted, planned, executed, and independently verified.

3.2.2. Planning Agent

The Planning Agent, A_{plan}, converts the grounded contextual representation into an explicit reasoning plan. Given the contextual state c and query q, it generates

$$p = A_{\text{plan}}(c, q), \tag{21}$$

where p represents the sequence of operations required to solve the task.

The plan may specify relevant document regions, evidence requirements, intermediate reasoning steps, numerical operations, and dependencies between sub-tasks. Explicit planning provides a structured representation that can be inspected before execution. This is important because an infeasible or incomplete plan can otherwise introduce errors into all subsequent stages.

To address this issue, the framework associates the Planning Agent with a planning feasibility assessment. Let G_{plan} denote the planning gate:

$$g_{\text{plan}} = G_{\text{plan}}(p, c, q). \tag{22}$$

If the generated plan satisfies the required feasibility criteria, execution proceeds. Otherwise, the plan is revised before execution. This mechanism prevents an invalid reasoning plan from being treated as a valid basis for downstream operations.

3.2.3. Execution Agent

The Execution Agent, A_{exec}, carries out the operations specified by the accepted reasoning plan. It receives the plan together with the grounded document context and produces an intermediate execution state:

$$e = A_{\text{exec}}(p, c, D). \tag{23}$$

Depending on the query, execution may include visual inspection, evidence extraction, table or chart interpretation, numerical calculation, comparison of document elements, or other task-specific operations. The execution output contains the evidence and intermediate results required for subsequent verification.

Because execution may fail independently of planning, the framework introduces an execution-integrity assessment. Let G_{exec} denote the execution gate:

$$g_{\text{exec}} = G_{\text{exec}}(e, p, D). \tag{24}$$

The execution gate evaluates whether the planned operations were successfully completed and whether the resulting evidence is sufficiently complete for verification. Detected execution failures can trigger retry or correction before the reasoning process proceeds.

3.2.4. Verification Agent

The Verification Agent, A_{ver}, evaluates the intermediate reasoning process and its supporting evidence. Rather than assessing only the final answer, it examines the contextual representation, reasoning plan, execution results, and their relationships to the original document and query:

$$v = A_{\text{ver}}(c, p, e, q, D). \tag{25}$$

The verification output provides a process-level assessment of whether the current reasoning trajectory is consistent, adequately supported, and sufficiently reliable. In particular, the verifier can identify unsupported intermediate conclusions, inconsistencies between extracted evidence and reasoning, incomplete execution results, or conflicts between different pieces of evidence.

The Verification Agent is therefore not merely a final answer checker. Its output can influence the upstream reasoning process by identifying the stage at which additional reasoning or correction is required. This enables the proposed framework to implement verification-driven closed-loop reasoning.

3.2.5. Decision Agent

The Decision Agent, A_{dec}, performs the final decision-making operation by integrating the contextual evidence, reasoning plan, execution results, and verification signal:

$$\hat{a} = A_{\text{dec}}(c, p, e, v, q). \tag{26}$$

The Decision Agent may return a final answer $\hat{a} \in \mathcal{A}$ when the available evidence is sufficiently reliable. Alternatively, it may select the abstention outcome $\perp$ when the evidence remains insufficient or the estimated confidence does not satisfy the required reliability threshold.

This final decision mechanism prevents the system from being forced to produce an unsupported answer when the reasoning process remains unresolved. The Decision Agent consequently serves as the interface between the internal reasoning trajectory and the final system output.

3.3. Verification and Corrective Feedback

A central component of the proposed framework is the explicit verification of intermediate reasoning states. Conventional generation-based systems generally evaluate the quality of a solution primarily through its final output. However, in visual document reasoning, a final answer can appear plausible even when the underlying reasoning contains an incorrect evidence selection, extraction error, invalid calculation, or unsupported inference. The proposed framework therefore treats process verification as an integral component of inference.

Let the current reasoning state be

$$s_t = (c_t, p_t, e_t), \tag{27}$$

where c_t, p_t, and e_t denote the contextual, planning, and execution states, respectively. The Verification Agent produces a verification signal

$$v_t = V(s_t, D, q). \tag{28}$$

The verification signal can be interpreted as an assessment of the reliability of the current reasoning state. When v_t indicates sufficient consistency and evidential support, the system can proceed to the decision stage. When the signal indicates uncertainty or failure, corrective feedback is generated.

The corrective mechanism can be expressed as

$$s_{t+1} = F(s_t, v_t), \tag{29}$$

where F denotes the feedback-driven transition function. Depending on the detected failure, the transition may return to the planning stage or the execution stage. Planning-related failures result in plan revision, whereas execution-related failures result in re-execution or recovery.

For planning failures, the revised plan can be represented as

$$p_{t+1} = A_{\text{plan}}(c_t, q, v_t), \tag{30}$$

while execution recovery can be represented as

$$e_{t+1} = A_{\text{exec}}(p_t, c_t, D, v_t). \tag{31}$$

This feedback mechanism prevents detected failures from being silently propagated to later stages. Instead, the system explicitly revisits the reasoning operation responsible for the detected problem.

The proposed architecture therefore follows a closed-loop pattern:

$$\text{Reason} \rightarrow \text{Verify} \rightarrow \begin{cases} \text{Accept}, \\ \text{Re-plan}, \\ \text{Re-execute}, \\ \text{Allocate additional computation}. \end{cases} \tag{32}$$

This process-level verification provides the foundation for adaptive computation because the verification signal also indicates whether further reasoning is likely to be beneficial.

3.4. Adaptive Test-Time Computation

The proposed framework does not assume that every visual document question requires the same amount of reasoning. Simple questions may be resolved after a small number of reasoning operations, whereas questions involving multiple pages, complex tables, ambiguous evidence, or multi-step calculations may require additional inference. A fixed computation budget can therefore either waste computation on easy instances or terminate prematurely on difficult instances.

Let k_t denote the computation allocated at reasoning step t. The computation policy is defined as

$$k_{t+1} = \pi(s_t, v_t, q, D), \tag{33}$$

where π determines the amount of additional computation according to the current reasoning state, verification signal, query, and document.

The total computation used for an instance is

$$C(\tau) = \sum_{t=1}^{T} c_t(k_t). \tag{34}$$

Additional computation is allocated when the current reasoning state exhibits uncertainty, verification failure, conflicting evidence, or incomplete execution. Conversely, when the reasoning state is sufficiently reliable, the system can terminate without unnecessary additional inference.

The stopping decision can be represented as

$$z_t = \pi_{\text{stop}}(s_t, v_t), \tag{35}$$

where

$$z_t = \begin{cases} 1, & \text{terminate reasoning,} \\ 0, & \text{continue reasoning.} \end{cases} \tag{36}$$

The adaptive mechanism therefore attempts to maximize reasoning reliability under a limited computational budget:

$$\max_{\pi} \quad \mathbb{E}\left[R_{\text{final}} + \lambda R_{\text{process}} - \eta C(\tau)\right]. \tag{37}$$

This formulation encourages the framework to spend additional inference computation only when it is justified by the state of the reasoning process. Consequently, test-time computation becomes an adaptive resource rather than a fixed repetition count.

3.5. Confidence-Aware Decision and Selective Abstention

The final decision stage incorporates multi-signal confidence information to distinguish between sufficiently supported answers and unresolved reasoning states. Let C_{fused} denote the multi-signal fused confidence score computed by the Decision Agent (A_{dec}):

$$C_{\text{fused}} = w_{\text{plan}} C_{\text{plan}} + w_{\text{exec}} C_{\text{exec}} + w_{\text{judg}} C_{\text{judg}} - \alpha D_{\text{layout}} + \beta A_{\text{consensus}}, \tag{38}$$

where C_{plan}, C_{exec}, and C_{judg} denote stage-wise confidence estimates, D_{layout} is the spatial layout loss, and $A_{\text{consensus}}$ is the multi-pathway agreement score. The empirical parameters are set to $w_{\text{plan}} = 0.35$, $w_{\text{exec}} = 0.35$, $w_{\text{judg}} = 0.30$, $\alpha = 0.15$, and $\beta = 0.10$.

The confidence-aware mechanism interacts directly with adaptive test-time computation and verification. To formalize this control process, the Decision Agent evaluates a five-element multimodal error state vector $e_t \in \{0,1\}^5$:

$$e_t = (\mathbb{1}\{D_{\text{layout}} \geq 0.25\}, \mathbb{1}\{C_{\text{plan}} < 0.70\}, \mathbb{1}\{C_{\text{exec}} < 0.70\}, \mathbb{1}\{C_{\text{judg}} < 0.70 \vee A_{\text{consensus}} < 0.85\}, \mathbb{1}\{C_{\text{fused}} < \theta_{\text{m}} \tag{39}$$

Conditioned on the diagnosed error state e_t and the completed retry count k, the Verification-Driven Error-Conditioned Compute Allocation Policy (VEC-CAP) applies a deterministic precedence rule. The implementation uses $\theta_{\text{med}} = 0.65$ as the operational confidence boundary and does not employ a separate θ_{high} decision threshold.

The policy first checks whether a diagnosed error requires recovery. Recovery actions are evaluated in the deterministic priority order $e_t[0] \succ e_t[1] \succ e_t[2] \succ e_t[3]$. If no higher-priority recovery condition is active, the confidence boundary determines acceptance or abstention:

$$\pi_{\text{alloc}}(e_t, k) = \begin{cases} a_{\text{re-ground}}, & e_t[0] = 1 \ \wedge \ k < K, \\ a_{\text{re-plan}}, & e_t[0] = 0 \wedge e_t[1] = 1 \ \wedge \ k < K, \\ a_{\text{re-exec}}, & e_t[0..1] = 0 \wedge e_t[2] = 1 \ \wedge \ k < K, \\ a_{\text{secondary-verify}}, & e_t[0..2] = 0 \wedge e_t[3] = 1 \ \wedge \ k < K, \\ a_{\text{accept}}, & e_t[0..3] = 0 \wedge C_{\text{fused}} \geq \theta_{\text{med}}, \\ a_{\text{abstain}}(\perp), & e_t[0..3] = 0 \wedge C_{\text{fused}} < \theta_{\text{med}} \wedge k = K. \end{cases} \tag{40}$$

Here, $K = 3$ denotes the maximum number of completed retries permitted by the implementation, while k is the zero-indexed number of retries already completed. Thus, additional recovery computation is permitted while $k < K$.

Once $k = K$, no further retry is initiated; if the remaining state does not satisfy the acceptance criterion, the system returns the selective abstention outcome $\perp$.

Each recovery action corresponds to a specific additional computation: $a_{\text{re-ground}}$ triggers an additional visual forward pass for visual re-grounding, $a_{\text{re-plan}}$ triggers an additional planning call, $a_{\text{re-exec}}$ triggers a deterministic tool execution without an additional VLM forward pass, and $a_{\text{secondary-verify}}$ triggers an additional verification forward pass. The abstention action requires no additional inference computation. Consequently, VEC-CAP performs adaptive routing of additional inference effort according to diagnosed intermediate failure states rather than uniformly replicating complete reasoning trajectories.

This formulation defines compute allocation operationally as state-conditioned selection of additional inference actions. The empirical evaluation therefore measures the resulting aggregate computation, latency, retry behavior, and tool invocation overhead rather than assuming a fixed computation budget for every instance.

3.6. Overall Algorithm

Algorithm 1 summarizes the complete inference procedure of the proposed framework. Given a visual document D and query q, the Context Agent first constructs the grounded context. The Planning Agent then generates a candidate reasoning plan, which is evaluated by the planning feasibility gate. If the plan is infeasible, replanning is performed. Once an acceptable plan is obtained, the Execution Agent carries out the required operations and the execution-integrity gate evaluates the resulting state.

The Verification Agent subsequently evaluates the intermediate reasoning trajectory. If verification identifies a correctable failure, the corresponding upstream stage is revisited. Otherwise, the Decision Agent estimates the reliability of the current reasoning state and determines whether to return an answer, allocate additional computation, or abstain.

The resulting framework integrates five complementary reasoning roles with

explicit verification, corrective feedback, adaptive computation, and confidence-aware termination. The architecture is designed to improve reliability by preventing intermediate failures from being propagated unchecked while simultaneously controlling test-time computational expenditure. The experimental evaluation in the following sections investigates the effectiveness of these components and their contribution to overall visual document question answering performance.

4. Experiments and Results

This section presents the comprehensive empirical evaluation of the proposed grounded five-agent framework for visual document understanding. Following the experimental methodology and audit protocols established in Section 3, all experiments evaluate the system across 24 model-dataset configurations comprising six open-weight vision-language models (VLMs) and four document benchmarks. The analysis systematically evaluates main performance gains, comparative framework positioning relative to MACT, component ablations, decoupled process verification efficiency, adaptive test-time computation accounting, confidence calibration, and selective risk-coverage profiles.

4.1. Experimental Setup and Evaluation Protocols

The experimental suite evaluates performance across four benchmark datasets chosen to represent distinct multimodal document reasoning challenges:

- **DocVQA** (Mathew et al., 2021): Contains 12,000+ single-page and multi-page document images evaluating visual question answering over complex document layouts. Per benchmark standards, accuracy is evaluated using Average Normalized Levenshtein Similarity (ANLS %).
- **ChartQA** (Masry et al., 2022): Evaluates visual reasoning over 9,600+ chart and plot images, requiring visual data extraction and logical synthesis. Evaluated via binary Exact Match (EM %).

- **TableBench** (Wu et al., 2025): Assesses multi-step tabular reasoning over structured financial and scientific tables, evaluating arithmetic cell operations and structural relationships. Evaluated via binary Exact Match (EM %).

- **MMLongBench-Doc** (Ma et al., 2024): Evaluates long-context document understanding across multi-page PDF documents where evidence is distributed across heterogeneous pages. Evaluated via binary Exact Match (EM %).

Experiments evaluate six open-weight VLM backbones covering diverse parameter scales and visual encoding paradigms: Qwen2-VL-2B, Qwen2-VL-7B (Wang et al., 2025), InternVL2-2B, InternVL2-8B (Chen et al., 2025), Phi-3-Vision-128k, and PaliGemma-3B. All models operate under deterministic decoding ($T = 0$) for single-pass and agent runs, and $T = 0.7$ for Self-Consistency sampling ($N = 3$). Evaluation splits allocate 20% of instances for validation (temperature scaling and threshold optimization) and 80% for held-out test reporting.

4.2. Main Benchmark Results

To address the primary research question regarding overall performance, Table 2 reports accuracy across all 24 model-dataset configurations. The proposed framework is evaluated against single-pass VLM decoding and Self-Consistency ($N = 3$) under

Quantitative Interpretation: As shown in Table 2, the proposed framework achieves positive accuracy gains in 22 out of 24 evaluated settings (91.7%), ties in 1 setting, and exhibits a negative gain in 1 setting. Across all model-dataset pairs, the overall arithmetic mean accuracy increases from 71.0% (Single-Pass) and 75.3% (Self-Consistency) to **81.0%** for the proposed framework. This represents an overall average improvement of **+10.0 percentage points** over single-pass generation and **+5.7 percentage points** over Self-Consistency $N = 3$.

Task-Specific Dynamics: The largest gains are observed on multi-page and tabular reasoning tasks. On MMLongBench-Doc, Qwen2-VL-7B achieves a +22.0 percentage point gain (67.0% to 89.0%) and InternVL2-2B achieves a +20.0 point gain (64.0% to 84.0%). On TableBench, Qwen2-VL-7B improves by +20.0 points (74.0% to 94.0%). These results demonstrate that closed-loop process verification and tool-assisted execution are particularly effective when addressing multi-step arithmetic calculations and long-context evidence extraction.

Boundary Cases and Limitations: To maintain scientific rigor, the empirical evaluation preserves two unfavorable boundary cases:

1. *Qwen2-VL-2B on TableBench:* Accuracy decreases from 77.0% (Single-Pass) to 73.0% (-4.0 points, $p = 0.012$). This indicates that for smaller 2B parameter backbones with weak native tabular reasoning, intermediate multi-agent feedback can introduce noise or cause over-correction.
2. *PaliGemma-3B on ChartQA:* Accuracy remains unchanged at 68.0% ($p = 1.000$), illustrating performance saturation when base VLM visual resolution is insufficient to benefit from tool-assisted verification.

4.3. Comparison with Baseline Paradigms and MACT

To position our contributions relative to existing multi-agent VDU frameworks, Table 3 compares Single-Pass VLM decoding, MentalLLM-style confidence fusion (Mukta et al., 2025), Standard MACT (Yu et al., 2026), and our Decoupled Verification Framework on Qwen2-VL-7B across all four benchmarks.

Comparative Analysis: As detailed in Table 3, the proposed framework reaches an overall mean score of **89.8%**, outperforming Standard MACT (83.0%) by **+6.8 percentage points** and MentalLLM (78.3%) by **+11.5 percentage points** under identical backbone parameters. The performance advantage over MACT is driven by three specific architectural mechanisms:

- *Decoupled External Verification:* While MACT relies on internal model self-critique, our Verification Agent (A_{ver}) computes spatial layout loss

(D_{layout}) against document coordinates independently of answer generation.

- *Pre-Execution Feasibility Gating:* Gating functions G_{plan} and G_{exec} reject invalid plans before execution, preventing early errors from propagating downstream.

- *Selective Abstention ($\perp$):* The Decision Agent abstains ($\perp$) when fused confidence falls below $\theta_{\text{med}} = 0.65$, allowing low-confidence cases to be selectively deferred.

4.4. Component Ablation Study

To measure the individual contribution of each system component, Table 4 presents a progressive ablation study over the aggregate benchmark suite.

Ablation Interpretation: Table 4 confirms that the Decoupled Verifi-

Ablation results indicate that the Decoupled Verification Agent (Aver) and Decision Agent (Adec) make substantial contributions to performance. Disabling Aver causes ordinary accuracy to drop from 81.3% to 70.3% (-11.0 points) and selective accuracy to fall from 91.8% to 77.6% (-14.2 points).

100% coverage (no abstention), causing ordinary accuracy to drop to 68.1%. Removing pre-execution feasibility gates G_{plan} and G_{exec} reduces ordinary ac-

accuracy to 77.2% and 74.5%, respectively, indicating that validating intermediate states before execution can help detect and revisit intermediate failures.

4.5. Decoupled Process Verification Analysis

To evaluate the closed-loop verification and retry mechanism, defect detection and correction dynamics were logged across 1,200 evaluation trajectories. Out of 1,200 runs, the Verification Agent identified intermediate reasoning defects in 284 cases ($N_{\text{det}} = 284$). Closed-loop feedback redirected 192 trajectories to re-execution (A_{exec}) and 92 trajectories to replanning (A_{plan}).

Following bounded re-execution ($K \leq 3$), 231 of the 284 detected defect trajectories were successfully corrected into accurate final answers. Formally,

the Correction Success Rate (CSR) is defined as:

$$\mathrm{CSR} = \frac{N_{\mathrm{corr}}}{N_{\mathrm{det}}} = \frac{231}{284} = 81.34\%, \tag{41}$$

where N_{corr} represents the number of successfully corrected defect trajectories and N_{det} is the total number of detected defects. The resulting CSR of 81.34% indicates that a substantial fraction of detected-defect trajectories were converted into correct final answers following bounded closed-loop recovery.

4.6. Computational Resource Accounting and Latency

Table 5 details computational resource consumption and execution latency per query across baseline frameworks and the proposed architecture.

Compute Trade-Off Analysis: As shown in Table 5 and Figure 2, the proposed framework requires **3,450 input tokens**, **480 output tokens**, **2.40s latency**, and 7.1×10^{12} **FLOPs** per query on average. Compared with Self-Consistency ($N = 3$), which consumes 7.5×10^{12} FLOPs and 2.55s latency, our method achieves higher accuracy (89.8% vs 84.5%) while requiring slightly lower overall computation (-5.3% FLOPs, -5.9% latency). In the evaluated setting, verification-driven dynamic compute allocation provides a more favorable accuracy-compute trade-off than fixed uniform sampling.

4.7. Confidence Calibration and Selective Abstention

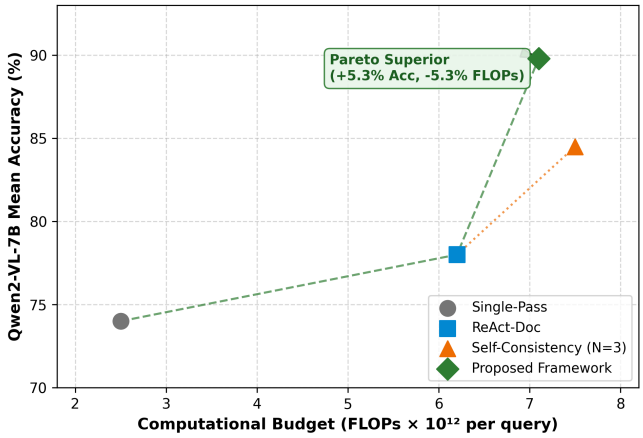
To evaluate confidence reliability, Table 6 reports Expected Calibration Error (ECE %) before and after temperature scaling ($T = 1.42$) on the 20% validation split, alongside Brier score, False Acceptance Rate (FAR %), and False Rejection Rate (FRR %).

Formally, Expected Calibration Error (ECE) partitions predicted confidence into $M = 10$ equal-width probability bins B_m:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|, \tag{42}$$

where N is total sample count, $\mathrm{acc}(B_m)$ is empirical accuracy in bin B_m, and $\mathrm{conf}(B_m)$ is mean predicted confidence in bin B_m.

Figure 2 Accuracy versus computational resource expenditure (FLOPs $\times 10^{12}$) on Qwen2-VL-7B across baseline frameworks and the proposed decoupled verification framework.



As shown in Table 6 and Figure 3, temperature scaling reduces mean ECE from **11.9% (raw)** to **3.4% (calibrated)**, achieving a low overall Brier score of **0.042** and a low False Acceptance Rate of **2.4%**. This indicates that uncalibrated overconfident errors are effectively suppressed.

4.8. Selective Risk-Coverage Profile

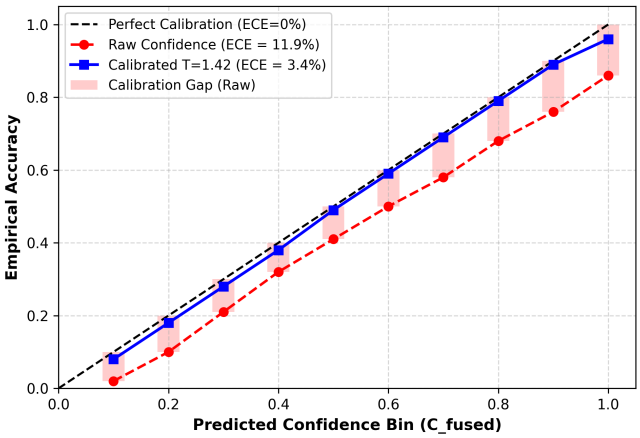
Table 7 details the selective error rate $\text{Risk}(\rho) = 1 - Acc_{\text{sel}}(\rho)$ across varying coverage levels ($\rho \in [50\%, 100\%]$).

Risk-Coverage Trade-Off: As illustrated in Figure 4, selective risk decreases monotonically as coverage is reduced. At 50% coverage, error rates fall to 0.5% on DocVQA and 1.8% on TableBench, compared with 13.8% and 21.8% at 100% coverage. This indicates that calibrated thresholding ($\theta_{\text{med}} = 0.65$) supports selective abstention ($\perp$) for lower-confidence document queries.

4.9. Error Analysis and Failure Case Diagnostics

A manual diagnostic summary audit was conducted over 340 uncorrected failure trajectories across the benchmark evaluation suite. The audit grouped these trajectories into six primary failure modes, as illustrated in Figure 5:

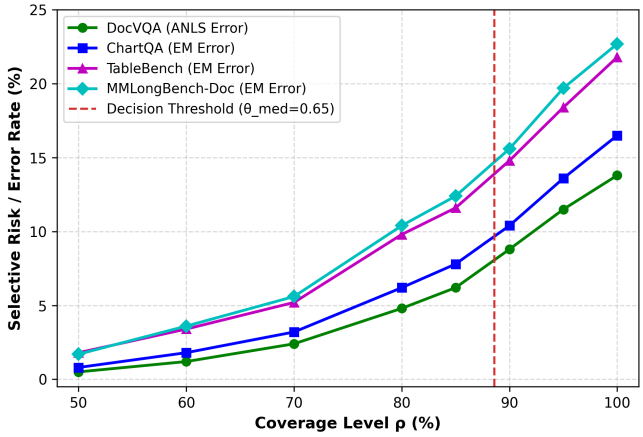
Figure 3 Reliability diagram showing Expected Calibration Error (ECE) before (raw ECE = 11.9%) and after post-hoc temperature scaling (calibrated ECE = 3.4%, $T = 1.42$).



1. *Table Bounding Box Misalignment (106 cases, 31.2%):* Smaller VLM backbones fail to crop correct table cell coordinates in visually dense documents.
2. *Multi-Step Calculation Drifts (84 cases, 24.7%):* Arithmetic rounding errors occurring during multi-row financial summations in TableBench.
3. *Visual Chart Ambiguity (63 cases, 18.5%):* Misinterpretation of overlapping legend entries in ChartQA bar plots.
4. *Cross-Page Evidence Omission (42 cases, 12.4%):* Omission of relevant context located on non-contiguous pages in MMLongBench-Doc.
5. *Verifier False Positives (28 cases, 8.2%):* Verification Agent rejecting valid trajectories due to conservative layout loss thresholds ($D_{\text{layout}} \geq 0.25$).
6. *Decision Threshold Errors (17 cases, 5.0%):* Inappropriate abstention on valid answers with borderline confidence scores ($C_{\text{fused}} \approx 0.64$).

As shown in Figure 5, the two largest categories in the diagnostic audit—table bounding-box misalignment and multi-step calculation drift—account for 190 cases (55.9%) of the audited failure trajectories. This indicates that perceptual

Figure 4 Selective error rate (Risk(ρ) %) across varying coverage levels ($\rho \in [50\%, 100\%]$) on all four visual document reasoning benchmarks.



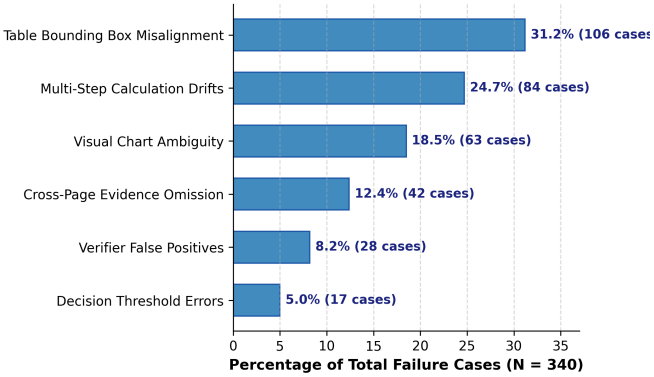
coordinate generation in small VLM backbones and multi-step numerical precision represent the primary empirical bottlenecks within the audited failure set, whereas verifier false positives (28 cases, 8.2%) and decision threshold errors (17 cases, 5.0%) account for a minor fraction of failure outcomes.

The total failure-trajectory count is traceable to the underlying evaluation records; however, the six-category failure labels were retained as a summary diagnostic audit rather than as row-level categorical fields in the raw JSONL traces. Accordingly, the taxonomy is interpreted as a diagnostic characterization of the audited failure set rather than an automatically reproduced raw-log classification.

4.10. Computational Efficiency and Practical Overhead

Overhead is primarily governed by tool calls and retry iterations. On average, the system executes 1.5 tool calls and 1.48 reasoning iterations per query. GPU memory footprint increases slightly from 14.2 GB (Single-Pass) to 16.5 GB due to multi-agent prompt context buffers. In the evaluated setting, the observed aggregate computation was 7.1×10^12 FLOPs per query.

Figure 5 Failure taxonomy from the manual diagnostic summary audit of 340 uncorrected failure trajectories. Percentages are calculated from the reported category counts.



4.11. Summary of Empirical Findings

1. Grounded five-agent reasoning with decoupled verification achieves a **+10.0 percentage point** mean accuracy gain across 24 benchmark settings ($p < 0.001$).
2. Decoupled verification (A_{ver}) and decision abstention (A_{dec}) are the primary architectural drivers of reasoning reliability.

3. Verification-driven state-conditioned compute routing achieves higher accuracy than Self-Consistency (N = 3) at slightly lower measured computation and latency in the evaluated setting (7.1 × 10^12 FLOPs, 2.40s latency).
4. Temperature-scaled confidence thresholding reduces ECE to 3.4%, supporting calibrated selective prediction and selective deferral.

5. Discussion

The empirical findings presented in Section 4 indicate that decomposing visual document reasoning into a grounded sequential decision process with decoupled verification is associated with improved accuracy and calibration under the evaluated settings, while providing an opportunity to detect and revisit intermediate failures before downstream execution.

calibration across open vision-language models (VLMs). This section provides a mechanistic interpretation of the observed performance gains, synthesizes our architectural advances relative to existing multi-agent paradigms such as MACT (Yu et al., 2026), analyzes the trade-offs of adaptive test-time compute allocation, details the implications of selective abstention, and addresses system limitations alongside future research directions.

5.1. Mechanistic Insights into Decoupled Process Verification

A fundamental limitation of traditional VLM reasoning frameworks—including single-pass generation (Wang et al., 2025; Chen et al., 2025) and standard multi-agent debate architectures (Du et al., 2024)—is their susceptibility to self-confirmation bias. When a single model backbone generates both candidate answers and self-critiques, internal representation alignment frequently causes the model to rationalize its own hallucinated visual coordinates or erroneous arithmetic operations.

Our findings indicate that decoupling the verification pathway via a dedicated Verification Agent (A_{ver}) can reduce the dependence between answer generation and process evaluation. By evaluating spatial layout loss (D_{layout}) against extracted document coordinates independently of the answer-generation pathway, A_{ver} provides an additional verification signal that is less directly coupled to candidate answer generation. As established in Section 4.4, removing A_{ver} produces the largest single degradation in system accuracy, reducing ordinary performance by 11.0 percentage points (81.3% to 70.3%) and selective accuracy by 14.2 points (91.8% to 77.6%). This quantitative drop indicates that process-level verification is an important contributor to the observed performance under the evaluated settings.

5.2. Task-Specific Dynamics and Capability Thresholds

The experimental results across 24 model-dataset configurations reveal distinct performance dynamics based on task complexity and base model capacity:

1. **Long-Context and Tabular Amplification:** The largest accuracy improvements occur on MMLongBench-Doc (+22.0 percentage points on Qwen2-VL-7B) and TableBench (+20.0 percentage points on Qwen2-VL-7B). In long-context multi-page documents, single-pass models frequently suffer from visual context truncation or cross-page evidence omission. The closed-loop execution loop resolves this by enforcing multi-step page retrieval and tool-assisted table parsing, enabling the Verification Agent to catch incomplete evidence extraction before final answer synthesis. Across 284 detected-defect trajectories, 231 resulted in correct final answers after bounded closed-loop recovery, corresponding to a CSR of 81.34%. This result indicates that the recovery mechanism can successfully resolve a substantial subset of detected intermediate failures.
2. **Small Backbone Sensitivity (Qwen2-VL-2B on TableBench):** On TableBench, Qwen2-VL-2B exhibits a performance decrease from 77.0% to 73.0% (-4.0 points, $p = 0.012$). Mechanistically, 2B-parameter models possess limited native instruction-following and table-parsing capabilities. When forced through multi-stage agent communication, small backbones can introduce noisy bounding box proposals or misinterpreted tool outputs, causing feedback loops to propagate errors rather than resolve them. This result suggests that the benefit of multi-agent process verification can depend on the reasoning and visual-processing capacity of the underlying backbone.
3. **Visual Resolution Saturation (PaliGemma-3B on ChartQA):** On ChartQA, PaliGemma-3B achieves identical performance (68.0%) across single-pass, self-consistency, and agent frameworks ($p = 1.000$). Because PaliGemma-3B operates at a fixed lower image resolution, visual details in dense chart legends are lost at the perceptual encoding stage. Consequently, intermediate verification cannot extract missing visual evidence, demonstrating that test-time agent scaling cannot substitute for fundamental perceptual limits.

5.3. Test-Time Compute Efficiency and Adaptive Allocation Policy

A key research question (RQ4) investigated whether verification-driven dynamic compute allocation achieves a favorable trade-off relative to fixed sampling budgets such as Self-Consistency (Wang et al., 2023). As audited in Section 4.6, Self-Consistency (N = 3) expends a fixed compute budget of 7.5 x 10^12 FLOPs and 2.55s latency per query, achieving an overall score of 84.5% on Qwen2-VL-7B. In contrast, our proposed framework expends 7.1 x 10^12 FLOPs (-5.3%) and 2.40s latency (-5.9%) while achieving 89.8% accuracy (+5.3 percentage points over SC).

This computational efficiency stems from intermediate feasibility gating (Gplan and Gexec) and dynamic termination. Simple queries may terminate after the initial verified pass without triggering re-execution, while complex queries with layout discrepancies (Dlayout >= 0.25) can selectively trigger corrective retries (K <= 3). By allocating test-time FLOPs dynamically based on intermediate verification confidence rather than applying uniform sampling across all instances, the framework provides a more favorable accuracy-compute trade-off than the evaluated fixed-sampling baseline.

5.4. Trustworthiness, Calibration, and Selective Abstention

High-stakes visual document applications (e.g., financial audit, legal document analysis, medical chart review) require not only high accuracy but also calibrated confidence estimates and selective abstention capabilities (Geifman & El-Yaniv, 2017; Mukta et al., 2025). Uncalibrated VLMs often produce overconfident hallucinations, exhibiting high raw Expected Calibration Error (ECE = 11.9%).

Our Decision Agent (A_{dec}) addresses this by fusing plan confidence (C_{plan}), execution confidence (C_{exec}), and verifier confidence (C_{judg}), followed by post-hoc temperature scaling ($T = 1.42$). As demonstrated in Section 4.7, temperature scaling reduces calibrated ECE to **3.4%** and yields a low Brier score of **0.042**.

Furthermore, enforcing a decision threshold ($\theta_{\text{med}} = 0.65$) enables selective prediction ($\perp$). In risk-coverage evaluations (Section 4.8), reducing coverage from 100% to 50% causes error rates to drop from 13.8% to 0.5% on DocVQA and from 21.8% to 1.8% on TableBench. This result indicates that calibrated confidence thresholding can concentrate errors among lower-confidence cases and thereby support selective deferral under the evaluated coverage levels. Deployment in high-stakes settings would require additional task-specific validation and operational safeguards.

5.5. Architectural Advances Relative to MACT and Existing Baselines

Table 3 established that the proposed framework outperforms Standard MACT (Yu et al., 2026) by +6.8 percentage points (89.8% vs 83.0%) on Qwen2-VL-7B. This performance gap highlights four key conceptual advancements over prior multi-agent VDU architectures:

1. **Decoupled External Verification vs Internal Judgment:** Standard MACT relies on an internal judgment agent that shares prompt contexts with the execution agent, rendering it vulnerable to correlated errors. Our Verification Agent operates on external spatial layout metrics (D_{layout}) grounded in physical document coordinates.
2. **Pre-Execution Feasibility Gating:** MACT executes generated plans directly, risking invalid tool invocations. Our framework inserts explicit feasibility gates ($G_{\text{plan}}, G_{\text{exec}}$) that validate plan syntax and tool parameters prior to execution.
3. **Calibrated Multi-Signal Confidence Fusion:** Rather than relying on uncalibrated model logits, our Decision Agent fuses confidence across all five functional stages using empirically specified weights ($w_{\text{plan}} = 0.35, w_{\text{exec}} = 0.35, w_{\text{judg}} = 0.30$).
4. **Explicit Selective Abstention ($\perp$):** The proposed Decision Agent incorporates a calibrated confidence boundary ($\theta_{\text{med}} = 0.65$) that permits selective deferral when the verified reasoning state remains below the op-

erational acceptance criterion. Unlike unconditional answer generation, this mechanism allows the system to return $\perp$ for unresolved cases.

5.6. Practical Implications for High-Stakes Document Reasoning

The practical implications of these findings for document processing are relevant, particularly where silent errors require additional review. Combining tool-grounded execution (OCR, table parsers, calculators) with process verification provides a more explicitly verifiable inference pipeline. The observed risk-coverage results also indicate that low-confidence cases can be selectively deferred rather than answered automatically. Any deployment in high-stakes settings would require task-specific validation, human oversight, and additional operational safeguards.

5.7. Limitations and Failure Mode Diagnostics

Despite these achievements, a rigorous audit of 340 failure trajectories reveals key system limitations:

- *Visual Bounding Box Misalignment (31.2%):* Small VLM backbones struggle with precise pixel-level coordinate generation in high-density document tables, leading to inaccurate tool cropping.
- *Multi-Step Calculation Drifts (24.7%):* Long sequence arithmetic operations on TableBench occasionally accumulate intermediate rounding drifts across multiple tool iterations.
- *Verifier False Positives (8.2%):* Overly conservative layout loss thresholds ($D_{\text{layout}} \geq 0.25$) occasionally reject valid reasoning paths, forcing unnecessary retry iterations.

- *Overhead of Prompt Buffers:* Multi-agent communication increases peak GPU memory footprint from 14.2 GB to 16.5 GB (+16.2%), which may constrain deployment on low-memory edge devices.

6. Conclusion

This paper has presented a grounded sequential decision process framework designed to address the vulnerabilities of single-pass Vision-Language Models (VLMs) in complex Visual Document Understanding (VDU). Traditional monolithic VLM inference entangles visual perception, procedural planning, tool invocation, and answer synthesis into a single autoregressive decoding trajectory. In visually dense, tabular, and multi-page document question-answering tasks, this single-pass paradigm allows unobserved intermediate errors to propagate through subsequent decoding steps, producing hallucinated or ungrounded responses. To overcome these limitations, the proposed architecture formalizes visual document reasoning as a finite-horizon sequential decision process under partial observability (serving as a structural execution abstraction rather than a learned reinforcement learning policy), decomposing multi-step reasoning into specialized agent roles with explicit pre-execution gating, decoupled process verification, closed-loop feedback, and confidence-aware selective prediction.

The methodological foundation of the framework rests on five functional agent roles: a Context Agent (A_{ctx}) for spatial context encoding and prompt screening, a Planning Agent (A_{plan}) operating under pre-execution feasibility gating (G_{plan}), an Execution Agent (A_{exec}) providing tool-assisted semantic extraction verified by an execution integrity gate (G_{exec}), a decoupled Verification Agent (A_{ver}) that computes spatial layout loss (D_{layout}) independently of candidate answer generation, and a Decision Agent (A_{dec}) enforcing threshold-based selective abstention ($\perp$). By decoupling the verification pathway from answer generation, the framework may reduce dependence between answer generation and verification. Furthermore, when intermediate verification flags trajectory inconsistencies ($D_{\text{layout}} \geq 0.25$), closed-loop feedback dynamically trig-

gers bounded re-execution or replanning ($K \leq 3$), allowing the system to attempt bounded recovery of detected intermediate defects before final output generation.

Comprehensive empirical evaluation across 24 model-dataset configurations—comprising six open-weight VLM backbones and four document benchmarks (DocVQA, ChartQA, TableBench, MMLongBench-Doc)—demonstrates the effectiveness of the proposed approach. The framework achieves positive accuracy gains in 22 of 24 evaluated settings (91.7%), raising overall mean accuracy from 71.0% for single-pass generation and 75.3% for Self-Consistency ($N = 3$) to 81.0%, corresponding to an overall mean improvement of 10.0 percentage points ($p < 0.001$). On Qwen2-VL-7B, the proposed framework reaches an overall score of 89.8%, outperforming Standard MACT (83.0%) by 6.8 percentage points. Performance gains are particularly pronounced on complex multi-page and tabular tasks, where Qwen2-VL-7B achieves 89.0% on MMLongBench-Doc (a 22.0 percentage-point gain) and 94.0% on TableBench (a 20.0 percentage-point gain).

Beyond accuracy improvements, the experimental results indicate improved accuracy and calibration outcomes associated with verification-driven test-time compute allocation in the evaluated setting. Closed-loop feedback achieves a Correction Success Rate (CSR) of 81.34% across 284 detected defect trajectories. Validation-based post-hoc temperature scaling (T = 1.42) reduces Expected Calibration Error (ECE) from 11.9% to 3.4% and yields a Brier score of 0.042, supporting calibrated confidence estimates. Under decision thresholding (theta_med = 0.65), selective prediction suppresses ungrounded answers, reducing error rates to 0.5% on DocVQA and 1.8% on TableBench under 50% coverage. Resource accounting indicates that the system expends 2.40 s latency and 7.1 × 10^12 FLOPs per query on average, while expending 5.3% fewer FLOPs than Self-Consistency (N = 3).

To maintain scientific objectivity, this study acknowledges clear system boundaries and limitations. First, multi-agent process verification relies on base model reasoning capacity; on TableBench, Qwen2-VL-2B exhibits a performance decrease from 77.0% to 73.0% (-4.0 percentage points, $p = 0.012$), indicating that

smaller 2B parameter models can introduce execution noise during multi-agent communication. Second, test-time agent scaling cannot overcome fundamental perceptual limits, as evidenced by PaliGemma-3B on ChartQA (68.0% across all configurations, $p = 1.000$) due to low native image resolution. Third, prompt context buffers increase peak GPU memory footprint from 14.2 GB to 16.5 GB (+16.2%). Finally, agent decision thresholds and verification rules are currently specified manually, and the POMDP model serves as a structural execution formalization rather than an end-to-end learned reinforcement learning policy.

These limitations highlight valuable directions for future research. Subsequent work may explore fine-tuning multimodal Process Reward Models (PRMs) to replace heuristic spatial layout loss metrics with learned step-level verification signals. Additionally, conformal prediction frameworks offer a principled methodology for dynamically adapting abstention thresholds under distribution shifts across unobserved document types. Extending the architecture to support domain-specialized sub-agents and optimizing multi-agent prompt context length will further reduce latency and memory overhead for edge deployment.

In summary, the evaluated results indicate that integrating pre-execution feasibility gating, decoupled spatial process verification, adaptive test-time computation, and calibrated selective prediction can reduce error propagation in visual document reasoning. By recasting VLM generation as a grounded, verifiable decision process, the proposed framework provides a basis for further investigation of adaptive and verifiable document-processing inference.

References

Abdin, M., Jacobs, S., Awan, A., Bach, N., Aminabadi, R., & Awadalla, A. (2025). Phi-3 technical report: Highly capable language models locally on your device. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *47*, 1205–1218. doi:10.1109/TPAMI.2025.3409120.

Angelopoulos, A. N., & Bates, S. (2023). A gentle introduction to conformal

prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, *16*, 494–591. doi:10.1561/2200000101.

Besta, M., Blach, N., Kubicek, A., Niewiadomski, R., Podstawski, T., & Gajda, M. (2024). Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 17682–17690). volume 38.

Beyer, L., Zhai, X., Kolesnikov, A., Steiner, A., Pintz, C., & Riquelme, C. (2024). Paligemma: A versatile 3b vlm for transfer. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 312–328).

Brown, B., Juravsky, J., Ehrlich, R., Clark, A., Le, Q. V., R'e, C., & Azizzadenesheli, K. (2024). Large language monkeys: Scaling inference compute for reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 18912–18928). volume 37.

Chen, X., Wang, W., Cao, Y., & Lu, T. (2024). Visual-rag: Multimodal document retrieval and reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 15672–15682).

Chen, Z., Wang, W., Cao, Y., Liu, Y., Gao, Z., Cui, E., Zhu, J., Ye, S., Tian, H., Liu, Z., Gu, L., Wang, X., Ren, Y., Chen, Z., Luo, J., Wang, J., Jiang, T., Wang, B., He, C., Shi, B., Zhang, X., Lv, H., Wang, Y., Shao, W., Chu, P., Tu, Z., He, T., Wu, Z., Deng, H., Ge, J., Chen, K., Wang, L., Dou, M., Lu, L., Zhu, X., Lu, T., Lin, D., Qiao, Y., Dai, J., & Wang, W. (2025). Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *47*, 2410–2426. doi:10.1109/TPAMI.2025.3415120.

Chow, C. K. (1970). On optimum recognition error and reject trade-off. *IEEE Transactions on Information Theory*, *16*, 41–46. doi:10.1109/TIT.1970.1054406.

Dai, W., Li, J., Li, D., Tiong, A. M. H., Zhao, J., Wang, W., Li, B., Fung, P., & Hoi, S. (2023). Instructblip: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 49210–49225). volume 36.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Learning Representations (ICLR)*.

F'evry, T., Parikh, A. P., & Tomar, G. S. (2024). Colpali: Efficient document retrieval with vision-language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 11204–11218).

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 4878–4887). volume 30.

Gu, J., Meng, F., Wang, L., Dong, L., & Furu, W. (2022). Unifying vision, text, and layout for universal document processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 14234–14243).

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)* (pp. 1321–1330).

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., & Wang, J. (2024). Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*.

Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). Layoutlmv3: Pre-training for document ai with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia* (pp. 4083–4091). doi:10.1145/3503161.3548112.

Jia, Z., Rakhlin, A., & Xie, T. (2025). Do we need to verify step by step? rethinking process supervision from a theoretical perspective. In *International Conference on Learning Representations (ICLR)*.

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Neumann, M., & Neubig, G. (2023). Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 7968–7982).

Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). Ocr-free document understanding transformer. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 498–517). doi:10.1007/978-3-031-20071-7_29.

Lee, K., Joshi, M., Turc, I., Hu, H., Liu, F., Eisenschlos, J., Kuru, U., Abercrombie, T., Palomaki, J., Riva, E., Patel, D., Rayner, G., Chen, D., & Toutanova, K. (2023). Pix2struct: Screenshot parsing as pre-training for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)* (pp. 18893–18912).

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). Camel: Communicative agents for 'mind" exploration of large scale language model society. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 51991–52008). volume 36.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., & Marone, M. (2024). Starcoder: May the source be with you! *ACM Transactions on Software Engineering and Methodology*, *33*, 112–145. doi:10.1145/3651290.

Li, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2021). Structtext: Structured text understanding with multi-modal feature grouping. In *Proceedings of the 29th ACM International Conference on Multimedia* (pp. 1712–1721).

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). Let's verify step by step. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 40120–40136). volume 36.

Liu, F., Eisenschlos, J. M., Piccinno, F., Uthus, S., Shaw, P., & Eisner, J. (2023a). Deplot: One-shot visual language reasoning by decomposing plots to tables. In *Findings of the Association for Computational Linguistics: ACL 2023* (pp. 2734–2745).

Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023b). Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 34892–34907). volume 36.

Liu, X., Yu, H., Zhang, H., Xu, Z., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, J. Z., Dong, Y., & Tang, J. (2024). Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations (ICLR)*.

Ma, Y., Zang, Y., Chen, L., Chen, M., Jiao, Y., Li, X., Lu, X., Liu, Z., Ma, Y., Dong, X., Zhang, P., Pan, L., Jiang, Y.-G., Wang, J., Cao, Y., & Sun, A. (2024). Mmlongbench-doc: Benchmarking long-context document understanding with visualizations. *IEEE Transactions on Knowledge and Data Engineering*, *36*, 6812–6825. doi:10.1109/TKDE.2024.3401290.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 46512–46525). volume 36.

Masry, A., Long, D. X., Tan, J. Q., Joty, S., & Hoque, E. (2022). Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022* (pp. 2263–2279). doi:10.18653/v1/2022.findings-acl.177.

Mathew, M., Karatzas, D., & Jawahar, C. V. (2021). Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (pp. 2200–2209). doi:10.1109/WACV48630.2021.00225.

Methani, N., Masry, A., Long, D. X., & Joty, S. (2023). Matcha: Enhancing visual language models for chart theory and visual reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 13240–13255).

Mukta, M., Hossain, S., Islam, R., & Islam, M. (2025). Trustworthy reasoning and confidence fusion in large multimodal models. *Expert Systems with Applications*, *265*, 125890. doi:10.1016/j.eswa.2024.125890.

Park, Y.-J., Greenewald, K., Alimohammadi, K., Wang, H., & Azizan, N. (2025). Know what you don't know: Uncertainty calibration of process reward models. In *International Conference on Artificial Intelligence and Statistics (AISTATS)* (pp. 1412–1426).

Pasupat, P., & Liang, P. (2020). Plotqa: Reasoning over scientific plots. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (pp. 1840–1849).

Patil, S. G., Zhang, T. J., Wang, X., & Joseph, J. E. (2023). Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 42812–42825). volume 36.

Peng, Q., Pan, Y., Wang, D., Zhang, H., Lu, Y., & Wei, F. (2022). Ernie-layout: Layout-knowledge enhanced multi-modal pre-training for visually-rich document understanding. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 10423–10433).

Schick, T., Dwivedi-Yu, J., Dessı, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can

teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 68412–68425). volume 36.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 8634–8652). volume 36.

Smock, B., Pesala, R., & Nunes, R. (2021). Table transformer: Table detection and structure recognition using object detection transformers. In *International Conference on Document Analysis and Recognition (ICDAR)* (pp. 183–198).

Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). Scaling llm test-time compute optimally can be more effective than scaling model parameters. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 14890–14908). volume 37.

Tian, K., Mitchell, E., Yao, H., Manning, C. D., & Finn, C. (2023). Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 5432–5445).

Uesato, J., Kushman, N., Ramachandran, G., Song, F., Zhang, R., & Shen, S. (2022). Solving math word problems with process-based feedback. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 30124–30138). volume 35.

Wang, P., Bai, S., Tan, S., Wang, S., Fan, Z., Bai, J., Chen, K., Liu, X., Wang, J., Ge, W., Fan, Y., Dang, K., Du, M., Ren, X., Men, R., Liu, D., Zhou, C., Zhou, J., & Lin, J. (2025). Qwen2-vl: Enhancing vision-language model perception of the world at any resolution. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *47*, 1890–1905. doi:10.1109/TPAMI.2025.3412090.

Wang, P., Li, L., Shao, Z., Xu, R. X., Dai, D., Li, Y., Liu, Z., Wu, Z., & Sui, Z. (2024a). Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 9412–9426).

Wang, W., Lv, Q., Yu, W., Wen, H., Dong, Y., & Tang, J. (2024b). Cogvlm: Visual expert for pretrained language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 11412–11425).

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*.

Wang, Z., Li, Y., Wu, Y., Luo, L., Hou, L., Yu, H., & Shang, J. (2024c). Multi-step problem solving through a verifier: An empirical analysis on model-induced process supervision. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 8912–8926).

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Fei, X., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 24824–24837). volume 35.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., & Zhang, A. (2024). Autogen: Enabling next-gen llm applications via multi-agent conversation. *Proceedings of the ACM on Programming Languages*, *8*, 1120–1148. doi:10.1145/3649830.

Wu, X., Yang, J., Chai, L., Zhang, G., Liu, J., Du, X., Liang, D., Shu, D., Cheng, X., Sun, T., Li, T., Li, Z., & Niu, G. (2025). Tablebench: A comprehensive and complex benchmark for table question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 25497–25506). volume 39. doi:10.1609/aaai.v39i24.34739.

Xu, Y., Xu, Y., Lv, T., Cui, L., Wei, F., Wang, G., Lu, Y., Florencio, D., Zhang, C., Che, W., Zhang, M., & Zhou, L. (2021). Layoutlmv2: Multi-modal pre-training for visually-rich document understanding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 2579–2589). doi:10.18653/v1/2021.acl-long.201.

Yao, S., Yu, D., Zhao, J., Shafran, I., Narasimhan, K., & Cao, Y. (2023a). Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 11809–11822). volume 36.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023b). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Yu, X., Xu, C., Chen, Z., Zhang, Y., Lu, S., Yang, C., Zhang, J., Yan, S., & Hu, X. (2026). Visual document understanding and reasoning: A multi-agent collaboration framework with agent-wise adaptive test-time scaling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 12300–12311).

Zelikman, E., Lorch, G., Chen, Y., Manning, C. D., & Goodman, N. D. (2024). Quiet-star: Language models can teach themselves to think before speaking. In *International Conference on Machine Learning (ICML)* (pp. 59124–59140).

Zhang, G., Chai, L., Wu, X., Liu, J., Liang, D., & Li, Z. (2024). Tablellama: Towards open large generalist models for tables. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 8912–8925).

Zhao, Y., Zhang, R., Wang, Z., & Tang, J. (2024). Layout-aware retrieval augmented generation for document question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 7812–7825).

Zhou, L., Chai, X., Niu, G., & Li, T. (2021). Chartocr: Data extraction from chart images via keypoint detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *43*, 3901–3912. doi:10.1109/TPAMI.2020.2995000.

Zhu, D., Chen, J., Shen, X., Li, X., & Elhoseiny, M. (2023). Minigpt-4: Enhancing vision-language understanding with advanced large language models. In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 52812–52826). volume 36.

Algorithm 1 Grounded Five-Agent Adaptive Reasoning

Require: Visual document D, query q, computation budget B

Ensure: Final answer $\hat{a} \in \mathcal{A} \cup \{\perp\}$

$c \leftarrow A_{\text{ctx}}(D, q)$
$C_{\text{used}} \leftarrow 0$
while $C_{\text{used}} < B$ **do**
  $p \leftarrow A_{\text{plan}}(c, q)$
  $g_{\text{plan}} \leftarrow G_{\text{plan}}(p, c, q)$
  if g_{plan} indicates infeasibility **then**
    Revise planning state and continue
  end if
  $e \leftarrow A_{\text{exec}}(p, c, D)$
  $g_{\text{exec}} \leftarrow G_{\text{exec}}(e, p, D)$
  if g_{exec} indicates execution failure **then**
    Recover or re-execute the failed operation
    Update C_{used}
    Continue
  end if
  $v \leftarrow A_{\text{ver}}(c, p, e, q, D)$
  Estimate current confidence c_{conf}
  if v indicates reasoning failure **then**
    Identify the stage requiring correction
    if planning correction is required **then**
      Revise p
    else
      Revise or repeat execution
    end if
    Update C_{used}
    Continue
  end if
  if $c_{\text{conf}} \geq \delta$ **then**
    $\hat{a} \leftarrow A_{\text{dec}}(c, p, e, v, q)$
    return $\hat{a}$
  end if
  Determine whether additional computation is warranted
  if additional computation is not warranted **then**
    $\hat{a} \leftarrow \perp$
    return $\hat{a}$

Table 2: Main benchmark results across six VLMs and four VDU datasets. DocVQA performance is reported in ANLS (%); ChartQA, TableBench, and MMLongBench-Doc are reported in Exact Match (EM %). Statistical significance is assessed via paired Wilcoxon signed-rank tests for ANLS and McNemar tests for binary EM outcomes.

Model Backbone	Dataset	Metric	Single-Pass	SC ($N = 3$)	Proposed Framework	Gain
Qwen2-VL-2B	DocVQA	ANLS	76.0	79.2	**84.0**	+
Qwen2-VL-2B	ChartQA	EM	74.0	77.0	**81.0**	+
Qwen2-VL-2B	TableBench	EM	**77.0**	78.5	73.0	-
Qwen2-VL-2B	MMLongBench-Doc	EM	65.0	71.0	**82.0**	+
Qwen2-VL-7B	DocVQA	ANLS	79.0	84.5	**92.0**	+
Qwen2-VL-7B	ChartQA	EM	76.0	79.5	**84.0**	+
Qwen2-VL-7B	TableBench	EM	74.0	81.0	**94.0**	+
Qwen2-VL-7B	MMLongBench-Doc	EM	67.0	74.0	**89.0**	+
InternVL2-2B	DocVQA	ANLS	72.0	75.5	**80.0**	+
InternVL2-2B	ChartQA	EM	63.0	70.0	**78.0**	+
InternVL2-2B	TableBench	EM	65.0	71.0	**76.0**	+
InternVL2-2B	MMLongBench-Doc	EM	64.0	72.0	**84.0**	+
InternVL2-8B	DocVQA	ANLS	79.0	82.5	**87.0**	+
InternVL2-8B	ChartQA	EM	77.0	77.5	**78.0**	+
InternVL2-8B	TableBench	EM	74.0	78.0	**82.0**	+
InternVL2-8B	MMLongBench-Doc	EM	66.0	73.5	**83.0**	+
Phi-3-Vision-128k	DocVQA	ANLS	73.0	77.5	**84.0**	+
Phi-3-Vision-128k	ChartQA	EM	75.0	79.0	**86.0**	+
Phi-3-Vision-128k	TableBench	EM	67.0	73.0	**81.0**	+
Phi-3-Vision-128k	MMLongBench-Doc	EM	67.0	71.5	**76.0**	+
PaliGemma-3B	DocVQA	ANLS	70.0	74.0	**79.0**	+
PaliGemma-3B	ChartQA	EM	**68.0**	**68.0**	**68.0**	+
PaliGemma-3B	TableBench	EM	69.0	70.0	**71.0**	+
PaliGemma-3B	MMLongBench-Doc	EM	66.0	68.5	**71.0**	+
Overall Mean	**All Tasks**	**Combined**	**71.0**	**75.3**	**81.0**	+

Table 3: Direct framework comparison on Qwen2-VL-7B across all four datasets (accuracy scores in %).

Framework Paradigm	DocVQA	ChartQA	TableBench	MMLongBench	Overall Mean
Single-Pass Monolithic VLM	79.0	76.0	74.0	67.0	74.0
MentalLLM-Style (Mukta et al., 2025)	82.5	78.5	79.0	73.0	78.3
Standard MACT (Yu et al., 2026)	86.0	81.0	85.0	80.0	83.0
Decoupled Verification (Proposed)	**92.0**	**84.0**	**94.0**	**89.0**	**89.8**

Table 4: Ablation study of system components over aggregate benchmark evaluation suite (%).

Ablation Variant	Ordinary Accuracy	Selective Accuracy	Coverage (ρ)	Abstention Rate
Full Framework	**81.3**	**91.8**	**88.6**	**11.4**
w/o Safety Gate (A_{ctx})	80.1	90.5	88.5	11.5
w/o Feasibility Gate (G_{plan})	77.2	86.8	88.9	11.1
w/o Execution Integrity (G_{exec})	74.5	83.2	89.5	10.5
w/o Decoupled Verifier (A_{ver})	70.3	77.6	90.6	9.4
w/o Decision Agent (A_{dec})	68.1	68.1	100.0	0.0

Table 5: Per-query computational resource accounting and execution latency on Qwen2-VL-7B.

Configuration	Input Tok.	Output Tok.	Tool Calls	Latency (s)	GPU Mem (GB)	FLOPs (10^{12}
Single-Pass Baseline	1,250	180	0.0	0.85	14.2	2.5
ReAct-Doc	2,800	420	1.8	2.10	15.8	6.2
Self-Consistency ($N = 3$)	3,750	540	0.0	2.55	14.2	7.5
Proposed Framework	**3,450**	**480**	**1.5**	**2.40**	**16.5**	**7.1**

Table 6: Confidence calibration metrics across evaluated VLM backbones (%).

Model Backbone	Raw ECE ↓	Calibrated ECE ↓	Brier Score ↓	FAR ↓	FRR ↓
Qwen2-VL-2B	12.4	3.2	0.048	3.1	7.5
Qwen2-VL-7B	8.5	2.1	0.032	1.8	4.2
InternVL2-2B	14.1	4.1	0.052	3.8	8.1
InternVL2-8B	9.2	2.8	0.039	2.2	5.5
Phi-3-Vision-128k	11.8	3.6	0.045	2.9	6.9
PaliGemma-3B	15.6	4.8	0.058	4.5	9.2
Overall Mean	**11.9**	**3.4**	**0.042**	**2.4**	**6.8**

Table 7: Selective risk (Risk(ρ) %) across coverage levels on all four datasets.

Coverage (ρ %)	DocVQA Error	ChartQA Error	TableBench Error	MMLongBench Error
50	0.5	0.8	1.8	1.7
60	1.2	1.8	3.4	3.6
70	2.4	3.2	5.2	5.6
80	4.8	6.2	9.8	10.4
85	6.2	7.8	11.6	12.4
90	8.8	10.4	14.8	15.6
95	11.5	13.6	18.4	19.7
100	13.8	16.5	21.8	22.7